

Lecture # 6

Multiple Activities and intents

By

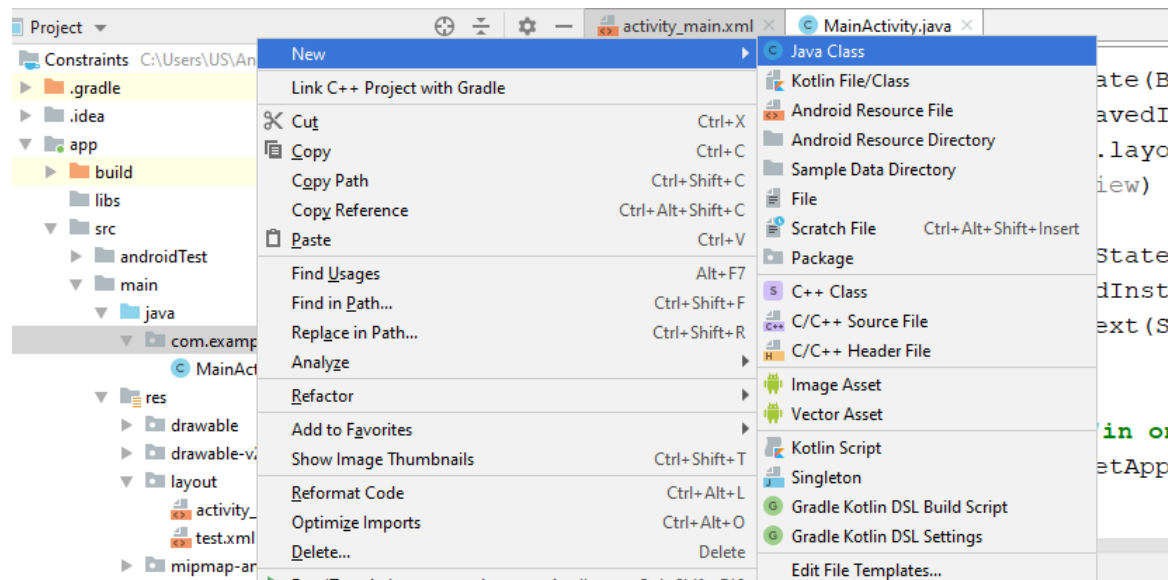
Dr. Muhammad Bilal

Today's Agenda

- Creating multiples activities and their respective layouts
- Initializing the new activity in the android manifest file
- Calling new activity using Intent and message passing

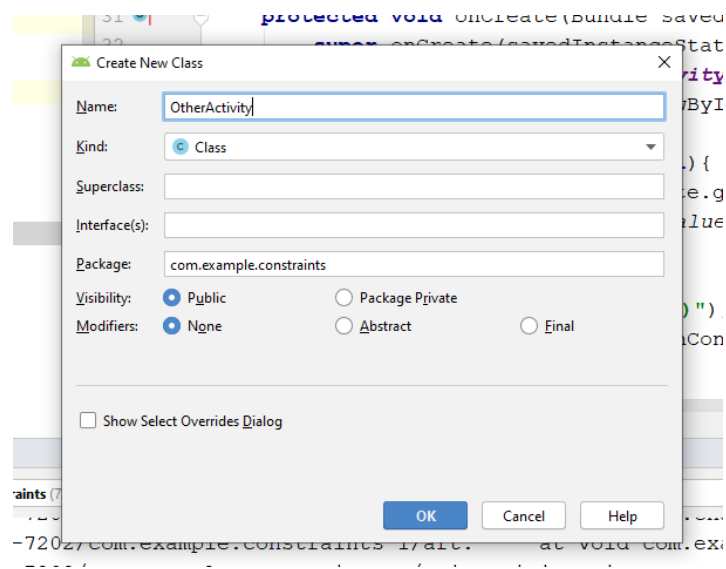
Creating Multiple Activities in Android App

- Generally many android apps have more than one activity to manage and display different aspects of apps and UI interfaces. E.g. Gmail, WhatsApp, and Facebook.
- To create another activity other than the main activity in android studio, just right-click on the package in java folder and create new java class



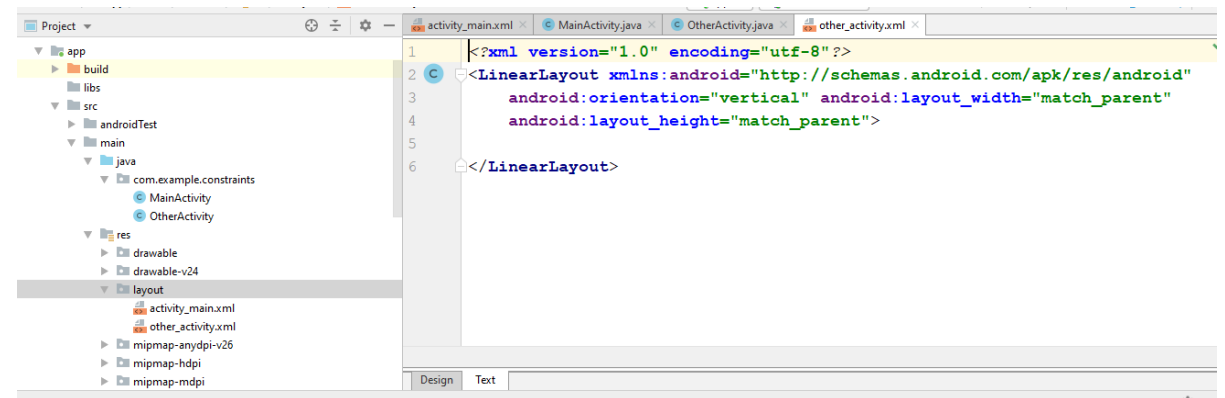
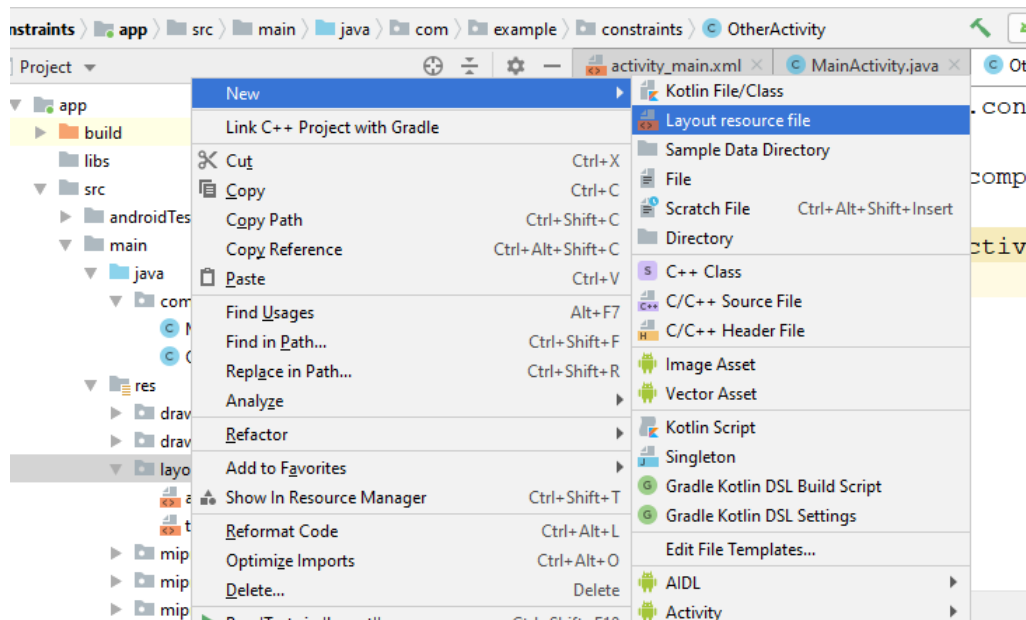
New Activity Configuration

- Set new activity name and press 'OK'
- The new activity is just a public class unless you extends it as "AppCompatActivity"



Creating New Activity XML Layout

- Create new layout in layout folder in 'res'
- Set new activity layout name and press 'OK' to get new layout



Set New Layout to New Activity

- Go back to newly created activity and create “onCreate” method by simply typing onCreate and select the onCreate method for “AppCompatActivity” to automatically add it.
- Now write setContentView to select content view by layout res ID.
- Add layout id in “setContentView” as parameter by writing “R.layout.other_avtivity”

```
public class OtherActivity extends AppCompatActivity {

    onCreate

}

public void onCreate(Bundle savedInstanceState) {
    protected void onCreate(Bundle savedInstanceState) {...} AppCompatActivity
    public boolean onCreateOptionsMenu()
    public CharSequence onCreateDescription()
    public View onCreatePanelView()
    public void onCreateContextMenu()
    public void onCreateNavigateUpTaskStack()
    public boolean onCreatePanelMenu()
    public View onCreateView()
    public View onCreateView()
    public void onCreateSupportNavigateUpTaskStack()
    constraints.D/MainActivity: In onStart()
    constraints.D/MainActivity: in onResume()

```

```
@Override  
protected void onCreate(@Nullable Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    setContentView(  
        m ↵ setContentTransitionManager(Transi... void  
        m ↵ setContentView(View view)           void  
        m ↵ setContentView(int layoutResID)      void  
        m ↵ setContentView(View view, LayoutPa... void  
        m ↵ setLocusContext(LocusId locusId, B... void  
    )  
}  
  
public class OtherActivity extends AppCompatActivity {  
  
    @Override  
    protected void onCreate(@Nullable Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.);  
    }  
}
```

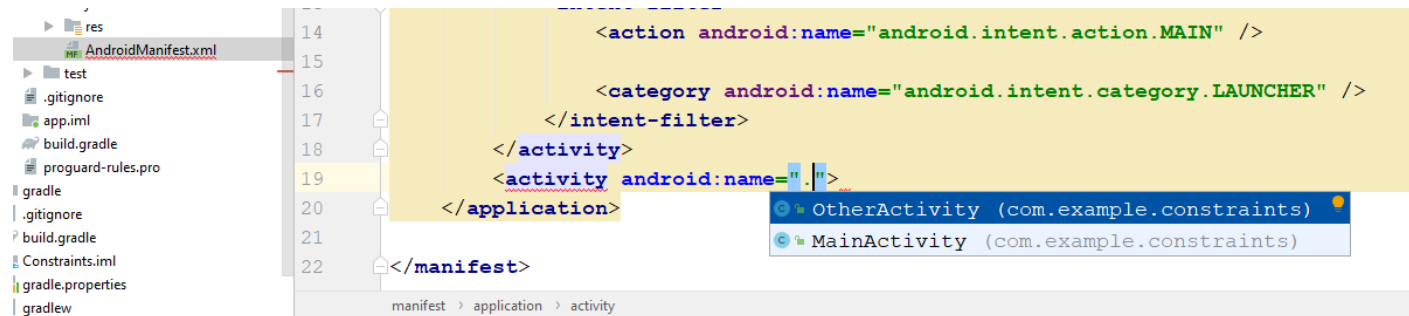
Adding New Activity to Android Manifest File

- Once the activity and its respective XML layout is created, the next step is to inform the android app about newly created activity
- To do this, open android manifest file “AndroidManifest.xml” in main folder.



Adding New Activity to Android Manifest File

- Under the “application” tab create new tab for “<activity>”, and set the name parameter with new activity starting with “.”
- End the tab using “</activity>” and save the android manifest file.



```
<activity android:name=".OtherActivity">
</activity>
</application>
```


Opening the New Activity

- More activities can be created using the same procedure in the android app.
- The new activity can be started and pass information between different activities using intents.

An intent is a type of message

- An **Intent** is a simple message object that is used to communicate and transfer between **android** components such as activities, content providers, broadcast receivers and services
- You can think of an intent as an “intent to do something.” It’s a type of message that allows you to bind separate objects (such as activities) together at runtime.
- If one activity wants to start a second activity, it does it by sending an intent to Android.
- Android will then start the second activity and pass it the intent.

```
Intent intent = new Intent(this, Target.class);
```

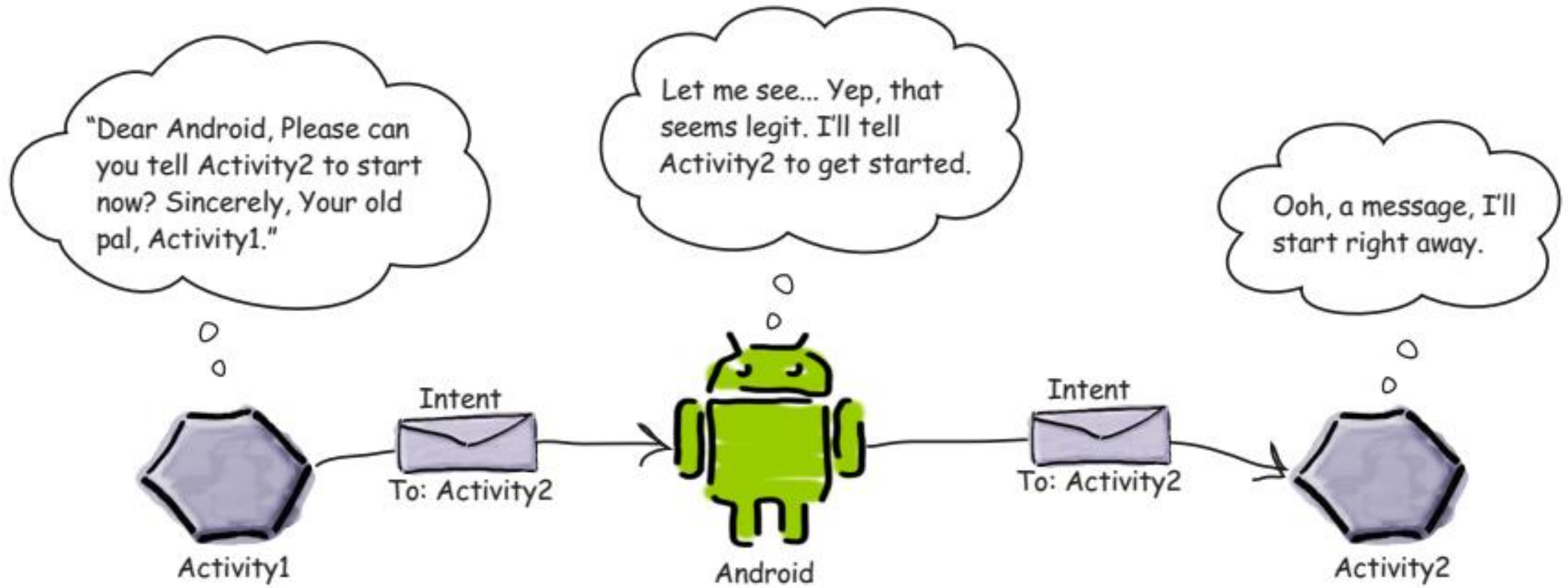
```
startActivity(intent);
```

startActivity() starts the activity specified in the intent..

The intent specifies the activity you want to receive it. It's like putting an address on an envelope.



To: AnotherActivity



Use an intent to start the second activity

```
package com.hfad.mymessenger;
```

```
import android.app.Activity;
```

```
import android.content.Intent; ← We need to import the Intent class  
import android.os.Bundle;      android.content.Intent as we're using  
import android.view.View;      it in onSendMessage().
```

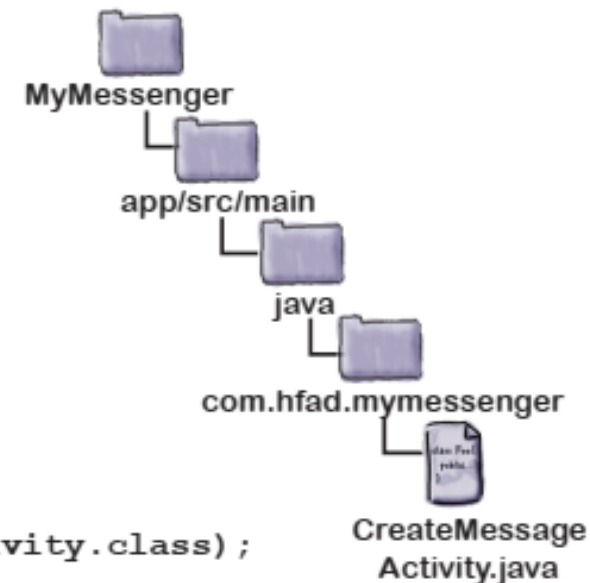
```
public class CreateMessageActivity extends Activity {
```

```
    @Override
```

```
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_create_message);  
    }
```

```
    //Call onSendMessage() when the button is clicked
```

```
    public void onSendMessage(View view) {  
        Intent intent = new Intent(this, ReceiveMessageActivity.class);  
        startActivity(intent); ← Start activity ReceiveMessageActivity.  
    }
```

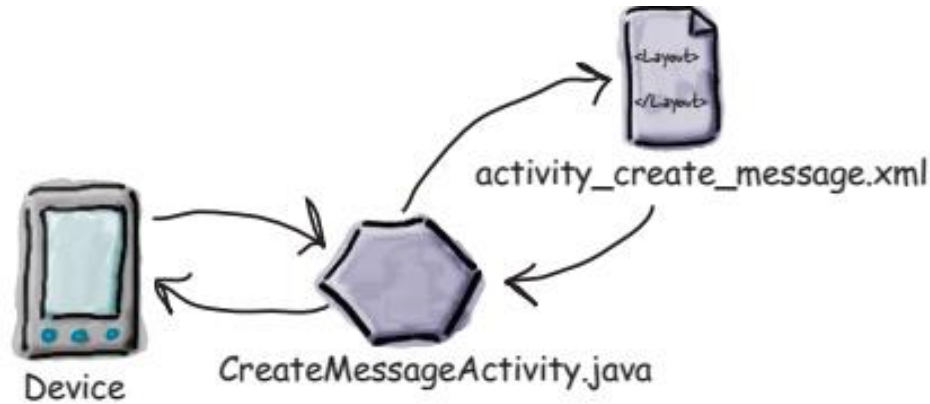


What happens when you run the app

1

When the app gets launched, the main activity, `CreateMessageActivity`, starts.

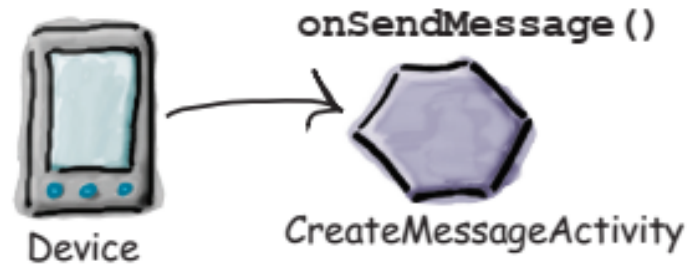
When it starts, the activity specifies that it uses layout `activity_create_message.xml`. This layout gets displayed in a new window.



2

The user types in a message and then clicks on the button.

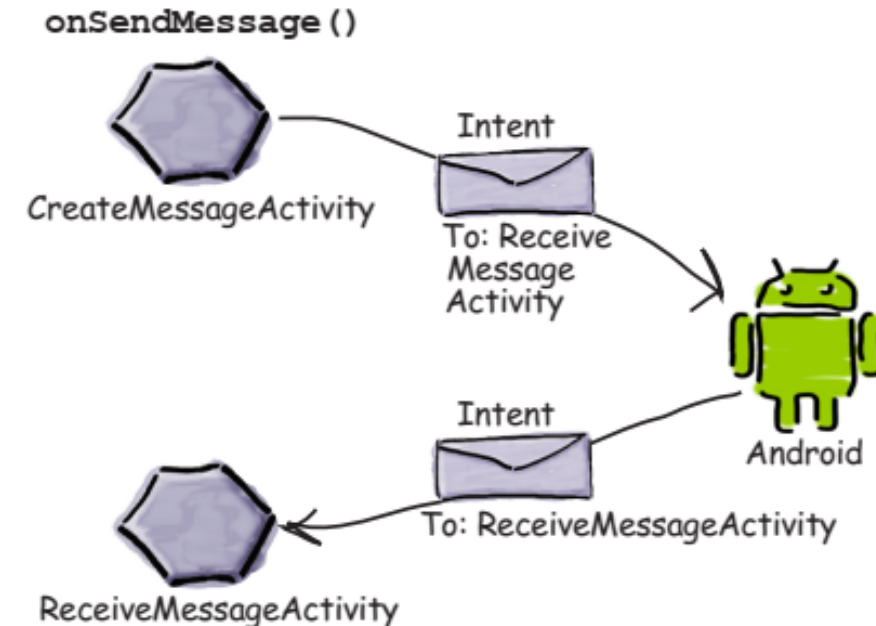
The `onSendMessage()` method in `CreateMessageActivity` responds to the click.



3

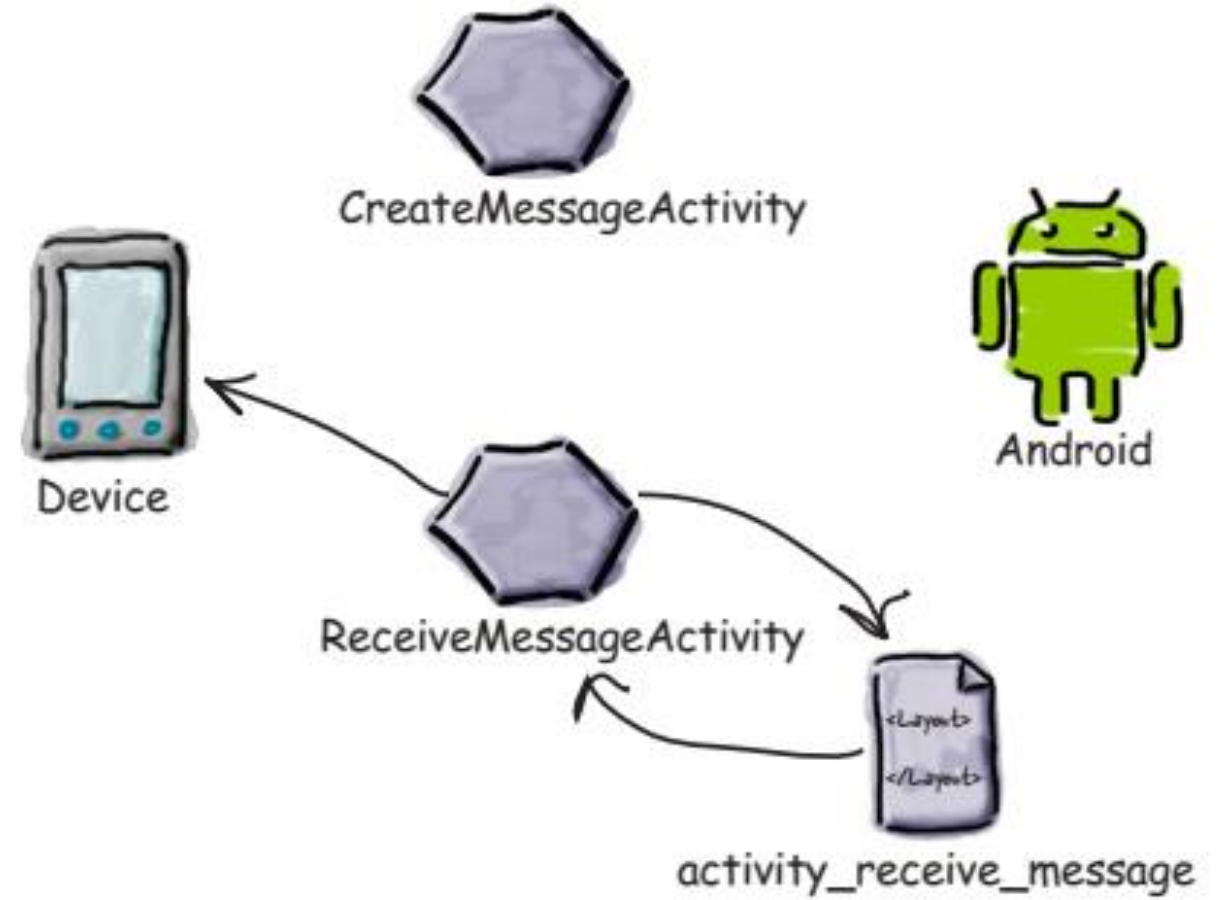
The `onSendMessage()` method uses an intent to tell Android to start activity `ReceiveMessageActivity`.

Android checks that the intent is valid, and then it tells `ReceiveMessageActivity` to start.



The story continues...

- 4 When `ReceiveMessageActivity` starts, it specifies that it uses layout `activity_receive_message.xml` and this layout gets displayed in a new window.



Pass text to a second activity

- `putExtra()` puts extra info in an intent
 - `Intent intent = new Intent(this, Target.class);`
 - `intent.putExtra("message", value);`
- Retrieve extra information from an intent

```
Intent intent = getIntent();
```

← Get the intent.

```
String string = intent.getStringExtra("message");
```

← Get the String passed along with the intent that has a name of "message".

- You're not just limited to retrieving String values. As an example, you can use:

```
int intNum = intent.getIntExtra("name", default_value);
```



Update the CreateMessageActivity code

```
import android.app.Activity;
import android.os.Bundle;
import android.content.Intent;
import android.view.View;
import android.widget.EditText;
//Call onSendMessage() when the button is clicked
public void onSendMessage(View view) {
    EditText messageView = (EditText)findViewById(R.id.message);
    String messageText = messageView.getText().toString();
    Intent intent = new Intent(this, ReceiveMessageActivity.class);
    intent.putExtra(ReceiveMessageActivity.EXTRA_MESSAGE, messageText);
    startActivity(intent);
}
```

You need to import the EditText class android.widget.EditText as you're using it in your activity code.

get the text that's in the EditText.

Start ReceiveMessageActivity with the intent.

Create an intent, then add the text to the intent. We're using a constant for the name of the extra information so that we know CreateMessageActivity and ReceiveMessageActivity are using the same String. We'll add this constant to ReceiveMessageActivity on the next page, so don't worry if Android Studio says it doesn't exist.

Get ReceiveMessageActivity to use the information in the intent

```
package com.hfad.mymessenger;
```

```
import android.app.Activity;
import android.os.Bundle;
import android.content.Intent;
import android.widget.TextView;
```

We need to import these classes.

Make sure your activity extends the Activity class.

```
public class ReceiveMessageActivity extends Activity {
```

```
    public static final String EXTRA_MESSAGE = "message";
```

This is the name of the extra value we're passing in the intent.

```
    @Override
```

```
    protected void onCreate(Bundle savedInstanceState) {
```

```
        super.onCreate(savedInstanceState);
```

```
        setContentView(R.layout.activity_receive_message);
```

```
        Intent intent = getIntent();
```

```
        String messageText = intent.getStringExtra(EXTRA_MESSAGE);
```

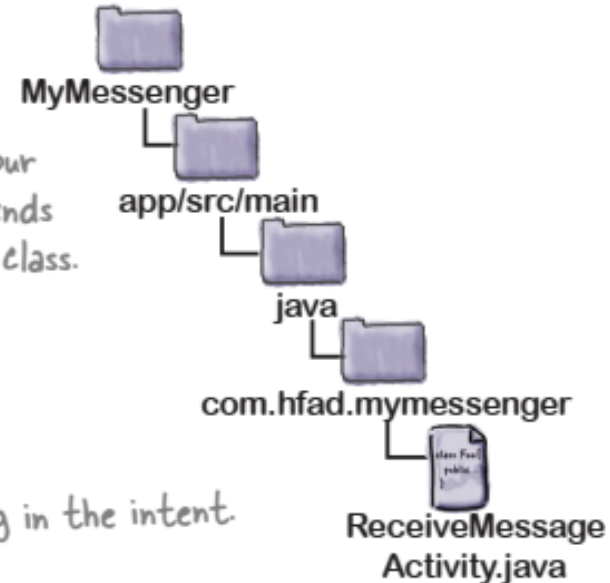
```
        TextView messageView = (TextView) findViewById(R.id.message);
```

```
        messageView.setText(messageText);
```

```
    }
```

```
}
```

Add the text to the message text view.



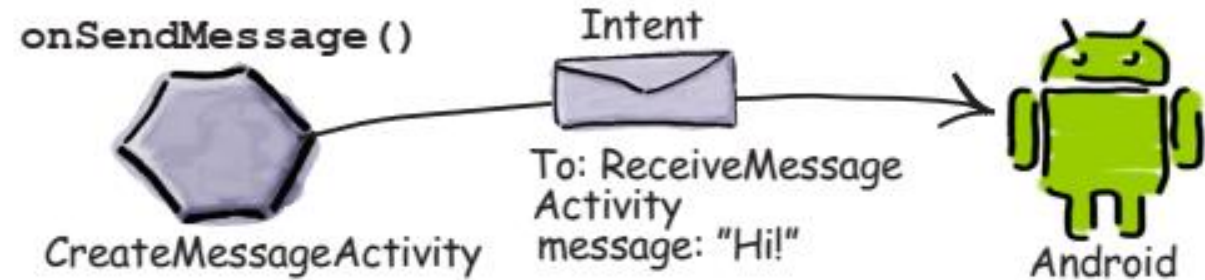
Get the intent, and get the message from it using `getStringExtra()`.

What happens when the user clicks the Send Message button

1

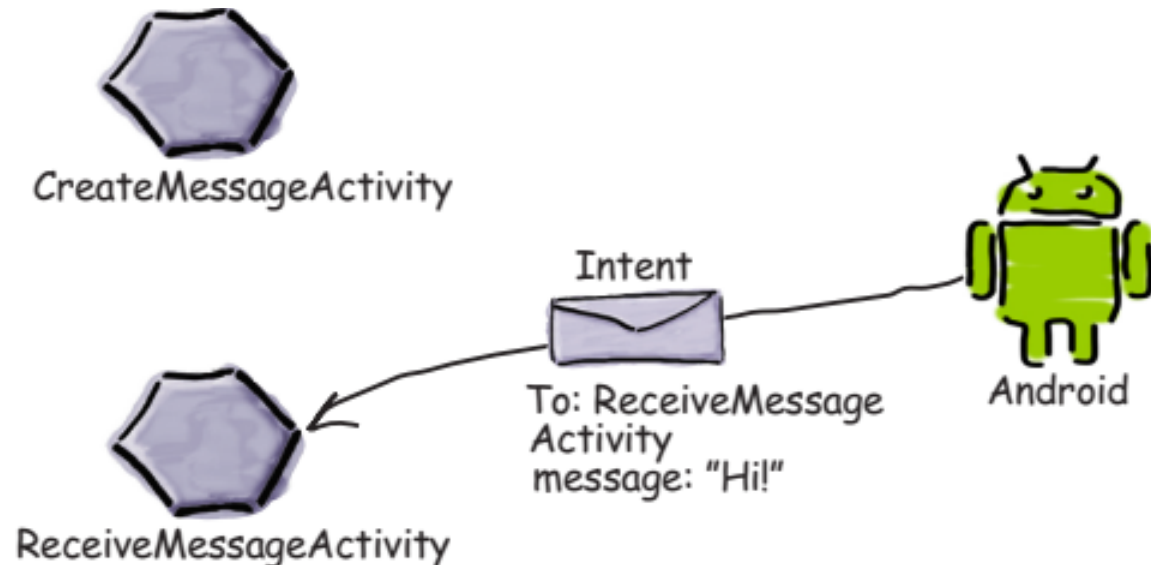
When the user clicks on the button, the `onSendMessage()` method is called.

Code within the `onSendMessage()` method creates an intent to start activity `ReceiveMessageActivity`, adds a message to the intent, and passes it to Android with an instruction to start the activity.



2

Android checks that the intent is OK, and then tells `ReceiveMessageActivity` to start.



3

When `ReceiveMessageActivity` starts, it specifies that it uses layout `activity_receive_message.xml`, and this gets displayed on the device.

The activity also updates the layout so that it displays the extra text included in the intent.

